Reinforcement Learning Lab

Lesson 5: Dyna-Q

Alberto Castellini, Riccardo Fidanza

University of Verona
email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2025-26

Environment Setup

The first step for the setup of the laboratory environment is to update the repository and load the miniconda environment.

Safe Procedure

Always back up the previous lessons' solutions before executing the repository update.

- Update the repository of the lab:

```
cd RL-Lab
git stash
git pull
git stash pop
```

- Activate the *miniconda* environment:

```
conda activate rl-lab
```

Today Assignment

In today's lesson, we implement the Dyna-Q and Dyna-Q+ algorithms in Python. In particular, the file to complete is:

```
RL-Lab/lessons/lesson_5_code.py
```

Inside the file, two functions are partially implemented. The objective of this lesson is to complete them.

- **def dynaQ()**
- **def dynaQplus()**

Expected results can be found in:

```
RL-Lab/results/lesson_5_results.txt
```

Algorithm: Dyna-Q

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:
$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Figure: Pseudocode for Dyna-Q, from the Sutton and Barto book *Reinforcement Learning: An Introduction*

Algorithm: Dyna-Q+

Dyna-Q+ encourages the exploration of recently unexplored transitions: given a reward R for a transition that has not been explored in τ time steps, during planning backups we replace the transition reward R with $R + k\sqrt{\tau}$, for some small k.
For further details and examples, refer to Chapter 8.3 from the Sutton and Barto book *Reinforcement Learning: An Introduction*, and try implementing Exercise 8.4.

Assignment Notes

Today's assignment is based on the *DangerousGridWorld* environment and makes use of the epsilon_greedy() function (provided).

Hint (Code)

The solutions of the previous lessons can be used to complete today's assignment. In particular, the update rule for the Q-table can be adapted from lesson 4 (Temporal difference methods) while the general structure follows lesson 3's solution (MC RL methods).

Results Disclaimer

Given the (high) stochasticity of the method, the obtained results may differ.

Hint (NumPy)

Numpy provides useful functions to simplify the assignment. In particular, numpy.random.choice() and numpy.where() are used in the suggested solution. More details can be found on the official website (here).